

RAG Memory Compactor

A self-hostable Python package that finds redundant chunks in a vector store, merges each redundant group into one summarized chunk, and archives the originals so every merge can be undone.

AUTHOR	STATUS	TESTS
Shantanu Somwanshi	Core pipeline complete	30 passing
SOURCE	LICENCE	REPOSITORY
~2,100 lines · 9 modules	MIT	github.com/ShantanuSomwanshi/rag-memory-compactor-and-eduplicator

CONTENTS

01 The problem

02 What the package does

03 The pipeline

04 Why two thresholds

05 Reversibility

06 Architecture

07 Swappable backends

08 Configuration

09 Command line

10 Measuring the benefit

11 Results so far

12 Testing

13 Build log

14 What remains

01 The problem

Vector databases bloat. The same material gets ingested twice, notes restate each other, a PDF overlaps a summary of itself, a document is re-uploaded after a small edit. Nothing detects this, so the store accumulates near-duplicate chunks indefinitely.

The cost is not mainly disk. It is that retrieval spends its top- k slots on near-copies. If three of your five retrieved chunks say the same thing in slightly different words, you have paid for five chunks of context and received three chunks' worth of information — and the two genuinely useful chunks that would have ranked fourth and fifth never made it into the prompt at all. That happens on *every single query*, forever.

So redundancy in a vector store is a recurring tax on both token spend and answer quality. This package is an attempt to collect that tax back.

02 What the package does

It runs over an existing vector store, identifies groups of chunks that genuinely duplicate each other, and replaces each group with a single LLM-written chunk that preserves every distinct fact from the originals. The originals are moved to an archive database rather than deleted, so any merge can be reversed.

Three properties were treated as requirements rather than nice-to-haves:

- **Self-hostable.** The whole pipeline runs locally — local vector store, local embedding model, and optionally a local LLM. Nothing has to leave the machine.
- **LLM-agnostic.** The summarization step routes through LiteLLM, so the model string is the only thing that changes between OpenAI, Anthropic, Groq, and a local Ollama model. No provider is privileged.
- **Reversible.** Compaction destroys information by design. That is only acceptable if it can be undone exactly, which drives the archive design in section 05.

03 The pipeline

Seven stages, run in order. The numbering is real — each stage consumes the previous stage's output.

01 Ingest and embed

Files are split on paragraph boundaries into chunks of ~900 characters with 120 characters of overlap; paragraphs longer than the target are hard-split with that overlap carried across the break. Each chunk gets a content-addressed id (SHA-256 over source, ordinal and text, truncated to 24 hex characters), so re-ingesting an unchanged file is idempotent. Chunks are embedded with `all-MiniLM-L6-v2` and stored in ChromaDB.

02 Candidate discovery

Every chunk queries the store for its `top_k` nearest neighbours. Pairs scoring at or above `similarity_threshold` (0.85) become edges in a graph, and each connected component of that graph becomes a candidate group. Components larger than `max_group_size` are split greedily, so one dense cluster cannot collapse a whole region of the store into a single chunk.

03 Validation gate

The full pairwise cosine similarity matrix of each candidate group is computed, and every pair must clear the stricter `validation_threshold` (0.90) before the group may be merged. Groups that fail are refined rather than discarded — see section 04. This is the step that separates the package from naive threshold deduplication.

04 Summarization

Only validated groups reach the LLM. The prompt instructs the model to rewrite the chunks as one self-contained chunk preserving every distinct fact, number, name and qualifier, to add nothing, and to make no reference to the merging process. Token usage is captured from the provider's response so the cost of compaction is measurable rather than assumed.

05 Replacement

The merged chunk is embedded and written to the store; the originals are deleted from it. The merged chunk carries provenance in its metadata — `merged_from` lists the source chunk ids, `merged_sources` the source files, plus the run id — so any chunk in the store can be traced back to what produced it.

06 Archiving

Before the originals leave the live store, each one is written to SQLite with its text, source, ordinal, metadata and its embedding vector as a BLOB. Storing the vector matters: an undo restores the exact original embedding rather than re-embedding, which would drift if the embedding model changed in between.

07 Undo

Undo deletes the merged chunk, restores the archived originals with their original vectors, and marks the merge as undone. It works at the level of a single merge or a whole run, and is idempotent — undoing twice is a no-op rather than an error.

04 Why two thresholds

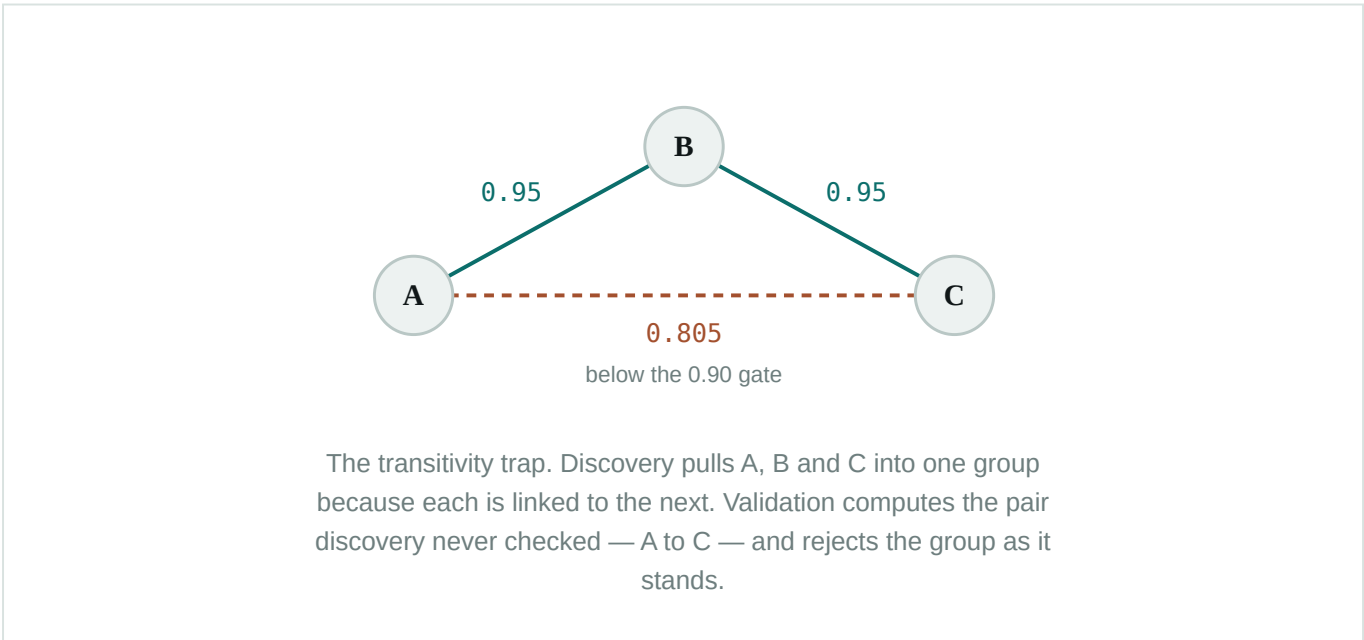
This is the central design decision, and the one worth defending closely.

Candidate discovery groups chunks by connected component, which links them *transitively*. If chunk A resembles B, and B resembles C, then all three land in one component — even though A and C were never directly compared and may not be alike at all. Cosine similarity is not transitive, so this happens routinely with a threshold low enough to catch real paraphrases.

Merging such a group summarizes unrelated material together and quietly loses information that retrieval can never recover. So discovery and validation deliberately use two separate thresholds with two different jobs:

THRESHOLD	DEFAULT	OPTIMIZES	JOB
similarity_threshold	0.85	Recall	Decides which chunks are worth <i>examining</i> as a possible duplicate group. Cheap — one ANN query per chunk.
validation_threshold	0.90	Precision	Decides which groups may actually be <i>merged</i> . Exact — the full pairwise matrix, every pair checked.

The configuration layer refuses to start if `validation_threshold` is set below `similarity_threshold`, since a validation pass looser than discovery would be no gate at all.



Refinement rather than rejection

Throwing the whole group away would waste a real duplicate pair. Instead, the package finds the largest subset in which every pair clears the threshold. That subset is, by definition, a maximal clique in the strict-similarity graph — and because groups are capped at `max_group_size` (8 by default), the clique can be computed exactly rather than approximated. In the diagram above, the refinement keeps `{A, B}`, drops C, and merges the tight core.

Every validation decision is reported with its reason, its weakest pair, and the ids it dropped, so a run is auditable rather than opaque.

05 Reversibility

Compaction is lossy by construction — several chunks become one. That is only safe to run unattended if the originals survive, which is why the archive is a first-class part of the design rather than a logging afterthought.

The archive is a SQLite database with three tables:

TABLE	HOLDS	KEY COLUMNS
runs	One row per compaction pass, with the full config serialized as JSON so a run's settings are recoverable.	run_id, created_at, config_json
merges	One row per merged group, with the text of the chunk that replaced it and whether it has been undone.	merge_id, run_id, merged_chunk_id, undone
originals	Every chunk that was replaced — full text, source, ordinal, metadata, and the embedding vector as a BLOB.	merge_id, chunk_id, text, vector

SQLite was chosen because it needs no server process and ships with Python — a self-hostable tool should not require the user to stand up a database to be able to undo something.

DESIGN CONSEQUENCE

Because vectors are archived alongside the text, undo is exact rather than approximate. Restoring by re-embedding would produce subtly different vectors if the embedding model or its version changed between the merge and the undo, which would silently alter retrieval behaviour after what the user was told was a full restore.

06 Architecture

Nine modules, roughly 2,100 lines. The layout mirrors the pipeline, so a reader can move from the stage list in section 03 to the code that implements it without a map.

MODULE	LINES	RESPONSIBILITY
<code>core.py</code>	346	Configuration (Pydantic), the data structures every stage passes around, and token accounting.
<code>ingest.py</code>	124	Loading text and PDF sources; paragraph-aware chunking.
<code>backends.py</code>	456	The three swappable protocols — embedding, vector storage, summarization — and every implementation of them.
<code>pipeline.py</code>	250	Candidate discovery and the validation gate: the propose-then-dispose pair.
<code>archive.py</code>	211	SQLite schema, merge recording, and the read path that undo depends on.
<code>compactor.py</code>	196	Orchestrator. Wires the stages over one store and archive pair; owns <code>compact</code> , <code>undo_merge</code> , <code>undo_run</code> .
<code>benchmark.py</code>	206	Corpus and retrieval token statistics, and the break-even calculation.
<code>cli.py</code>	261	Typer command line.
<code>__init__.py</code>	50	Public surface, with a lazy re-export so importing the package stays cheap.

Every heavy dependency — `chromadb`, `sentence-transformers`, `litellm`, `tiktoken`, `pypdf` — is imported lazily inside the class that needs it, never at module top level. That is what keeps them genuinely optional extras: the package imports, and the entire test suite runs, with none of them installed.

07 Swappable backends

Three protocols define the seams. The pipeline talks only to these, so any implementation can be substituted without touching pipeline code — which is what "LLM-agnostic" and "self-hostable" actually reduce to in practice.

PROTOCOL	IMPLEMENTATIONS	NOTES
Embedder	SentenceTransformerEmbedder HashingEmbedder	The hashing embedder uses the hashing trick over word unigrams and bigrams with signed buckets. It is deterministic, instant and offline — not competitive with a trained model, but it makes tests, CI and demos runnable with no model download.
VectorStore	ChromaStore InMemoryStore	The in-memory store does exhaustive rather than approximate search, which is the right trade at this scale and makes test results deterministic. All vectors are L2-normalized on write, so cosine similarity is a plain dot product everywhere downstream.
Summarizer	LiteLLMSummarizer StubSummarizer	LiteLLM handles provider routing, so the model string is the only variable. The stub is extractive — it keeps the longest chunk as a spine and adds only sentences that are not near-repeats (word-level Jaccard below 0.6) — which lets the pipeline run end to end with no key and no network.

Choosing a model

```

"llm_model": "gpt-4o-mini"           // OPENAI_API_KEY
"llm_model": "claude-3-5-haiku-20241022" // ANTHROPIC_API_KEY
"llm_model": "groq/llama-3.1-8b-instant" // GROQ_API_KEY
"llm_model": "ollama/llama3.2:3b"      // fully local, no key

```

When a provider does not report token usage, the summarizer falls back to counting locally rather than recording zero — otherwise the compaction cost in the benchmark would be understated.

08 Configuration

Settings come from `ragcompactor.json`, overridden by `RAGCOMPACTOR_*` environment variables. The config model is Pydantic, so invalid combinations fail at load rather than mid-run.

KEY	DEFAULT	MEANING
store_backend	memory	<code>chroma</code> for a persistent store, <code>memory</code> for in-process
embedding_backend	sentence-transformers	or <code>hashing</code> for offline operation
embedding_model	all-MiniLM-L6-v2	Any sentence-transformers checkpoint
chunk_size	900	Target chunk size in characters
chunk_overlap	120	Overlap carried across hard splits
top_k	10	Neighbours examined per chunk during discovery
similarity_threshold	0.85	Candidate discovery gate (recall)
validation_threshold	0.90	Merge gate (precision); must be $\geq$ discovery
max_group_size	8	Cap on chunks merged into one summary
refine_rejected_groups	true	Keep the largest valid subset of a failing group
llm_backend	litellm	or <code>stub</code> for offline runs
llm_model	gpt-4o-mini	Any LiteLLM model string
max_summary_tokens	512	Ceiling on the merged chunk

09 Command line

```

pip install -e ".[all]"          # chroma, sentence-transformers, litellm, tiktoken, pypdf

ragcompactor demo                # offline end-to-end: seed, compact, undo
ragcompactor init                # write a starter config
ragcompactor ingest ./notes ./papers
ragcompactor compact --dry-run   # what would merge; no LLM call, no writes
ragcompactor compact
ragcompactor runs                # list compaction runs
ragcompactor stats               # store and archive counts
ragcompactor undo --last
ragcompactor benchmark --queries queries.txt --top-k 5

```

`demo` builds a small synthetic corpus, compacts it and undoes the run using the hashing embedder, in-memory store and stub summarizer — no API key, no model download, no network. It is also seeded so that the validation gate visibly refines an over-broad group rather than merging it whole.

`compact --dry-run` reports what would merge without calling the LLM or writing anything, so thresholds can be tuned against a real corpus for free before any tokens are spent.

10 Measuring the benefit

The benchmark is deliberately constructed to resist the obvious objections, because the easy version of this measurement is misleading.

What it compares

The same corpus, the same queries, the same top-*k* and the same embedding model — differing only in whether compaction ran. Anything else measures something other than this tool.

WHAT IT DELIBERATELY DOES NOT DO

It does not compare against stuffing raw files into a prompt. Any RAG setup beats that, so such a comparison would credit compaction for savings that plain retrieval already provides, and would inflate the headline number while proving nothing about deduplication.

It also does not hide the cost. Summarization spends tokens. The report states the one-off compaction cost and the number of queries needed to repay it. If compaction did not shrink retrieved context, break-even reads `never` rather than being quietly omitted.

What it reports

- **Storage.** Chunk count and total corpus tokens, before and after — the one-time win.
- **Per-query context.** Tokens pulled into the prompt at identical top-*k* — the recurring win, and the one that compounds.
- **Break-even.** Compaction cost divided by tokens saved per query: the number of queries after which the tool is ahead.

Token counting uses `tiktoken` where available. Where its vocabulary cannot be loaded it falls back to a characters-per-token estimate, and the report says so explicitly rather than presenting an estimate as a measurement.

11 Results so far

From an offline run on a small synthetic corpus containing known paraphrases, using the stub summarizer:

CORPUS CHUNKS	CORPUS TOKENS	CONTEXT PER QUERY
6 → 5	206 → 164	113.3 → 98.5
1 removed	20.4% smaller	13.1% smaller

The `demo` command independently shows 9 chunks compacting to 7 across two validated groups, with `undo` restoring all 9 exactly.

HOW TO READ THESE NUMBERS

These come from the extractive stub, which can only select sentences and never rewrite them, so they understate what a real model achieves. They are also from a corpus built to contain duplicates. The honest claim is conditional — "on a corpus with this much redundancy, context per query fell by this much" — not an unqualified percentage. A run against a hosted model on a real corpus is still outstanding.

12 Testing

Thirty tests, all passing, all running fully offline via the hashing embedder, in-memory store and stub summarizer. No network, no API key, no model download — which means the suite runs in CI and on a machine that has never been configured.

AREA	WHAT IS VERIFIED
Chunking	Whitespace normalization, paragraph packing within the size limit, hard-splitting of oversized paragraphs with overlap, content-addressed ids stable across repeat calls, directory walks de-duplicating exact repeats.
Discovery	Duplicates group together while genuinely distinct chunks stay ungrouped; transitive chaining reproduced deliberately; <code>max_group_size</code> caps merge width.
Validation	A transitive group is rejected under a strict gate and refined to its valid core when refinement is enabled; a genuinely tight pair passes untouched; a group with no valid pair is rejected outright; clique selection returns the true largest subset.
Compaction	Duplicates merge and distinct chunks survive; provenance metadata is correct; <code>--dry-run</code> writes nothing and records no run; compaction is idempotent on an already-clean store.
Archive and undo	Undo restores exact original text and count; repeat undo is a no-op; unknown merge ids are reported rather than raising; run and archive statistics are accurate.
Benchmark	Break-even divides cost by per-query saving correctly; a zero-saving run reports <code>never</code> ; a zero-cost stub run is flagged as not meaningful; token counting is positive and monotonic.

13 Build log

What was decided and built, in order, with the reasoning that drove each step.

Concept	Problem framed as vector bloat. Redundant chunks accumulate in a local vector store; fix it by detecting and merging them rather than by asking users to curate their sources.
---------	---

Design	Pipeline settled. Ingest, embed into ChromaDB, ANN top- <i>k</i> plus similarity mapping, LLM summarization, replacement, archive, undo. A validation step using cosine similarity was then added ahead of summarization, to confirm that grouped chunks really are similar enough before anything is merged.
--------	--

Design	Positioned as a distributable package , not an internal script: self-hostable, LLM-agnostic, aimed at cutting token usage and storage footprint.
--------	---

Decision	LiteLLM-style pluggable backends chosen for the LLM seam, over an OpenAI-compatible-endpoint abstraction or a hand-rolled one — so "works with any LLM" would not quietly mean "tested only against Ollama".
----------	---

Decision	Summarization routed to a hosted API rather than local Ollama for development, with local models remaining one configured backend among several.
----------	---

7 Sept	Scaffold and core pipeline built — package layout, config, data models, ingestion, embedding, both stores, discovery, validation, both summarizers, archive, orchestrator, benchmark, CLI, and a 30-test suite. All tests green.
--------	---

7 Sept	A benchmark run exposed a real bug. The first stub summarizer concatenated chunks rather than compressing them, so the measured context per query went <i>up</i> after compaction. The benchmark correctly reported a negative saving — the tooling was working. The stub was rewritten to be extractive, with Jaccard-based sentence deduplication, after which the same corpus showed a genuine 20% corpus reduction.
--------	--

7 Sept	Token counting hardened. Where <code>tiktoken</code> 's vocabulary cannot be downloaded, the report now distinguishes "estimate" from "measurement" instead of silently presenting one as the other.
--------	---

-
- 7 Sept **Published to GitHub** — repository initialized, 24 files committed, pushed.
-
- 8 Sept **Module layout consolidated** from 14 files to 9, on the grounds that several boundaries were finer than the project needed. Groupings were chosen to mean something: `backends.py` is everything behind a swappable protocol, `pipeline.py` is the propose-then-dispose pair, `core.py` is the plain-data layer. Verified afterwards that all modules import cleanly with no cycles and all 30 tests still pass.
-
- 8 Sept **Demonstration methodology worked out** — the A/B design, the three levels of measurement, the break-even framing, and a planned redundancy sweep that would show savings rising with duplication and falling to near zero on a clean corpus.
-

14 What remains

OPEN **Retrieval-quality evaluation.** The package can currently show that the store got smaller and cheaper, but not that answers stayed as good. This is the sharpest available criticism — efficiency is indistinguishable from data loss without it, since any corpus can be made smaller by deleting it. A set of questions with known correct source chunks, measured before and after compaction, would close it.

OPEN **A run against a real model.** Every number so far comes from the offline stub. A hosted-model run on a real corpus would produce both honest figures and the first true compaction cost.

OPEN **End-to-end ChromaDB run.** Everything has been exercised through the in-memory store; the persistent path needs one real run before a live demo.

OPEN **A/B and redundancy-sweep harness.** Designed but not built — the script that produces the comparison figures and the savings-versus-redundancy curve.

DONE **Core pipeline, tests, packaging and publication.** All seven stages implemented and verified, the package installable, the repository public.

Where the effort should go

The retrieval-quality evaluation is worth more than any additional feature. A reviewer will accept a modest, well-measured saving; what they will not accept is a compression claim with no evidence that the compressed store still answers correctly.

